

A short primer on retrieval

Why one retriever is not enough

A dense retriever encodes a query and every chunk into the same vector space and compares them by cosine similarity. It is good at meaning. Ask it about "employee turnover" and it will surface a passage about "staff attrition" even though the two share no words.

It is also, reliably, bad at exact terms. Product codes, surnames, statute numbers, error strings and version identifiers are precisely the tokens an embedding model has the least reason to preserve, because they carry almost no distributional meaning. A query for "CVE-2021-44228" against a dense index will often return passages about vulnerabilities in general and miss the one page that names it.

A sparse retriever has the opposite profile. BM25 scores a document by how often the query terms appear in it, adjusted for how rare those terms are across the collection and how long the document is. It finds "CVE-2021-44228" instantly. It has no idea that turnover and attrition are the same thing.

Reciprocal rank fusion

The naive way to combine the two is to concatenate their result lists and remove duplicates. This does not work well, for a reason that is easy to miss: it throws away agreement. If both retrievers independently rank the same chunk near the top, that chunk is far more likely to be relevant than one that only a single retriever liked, and concatenation gives it no credit at all.

Reciprocal rank fusion gives each chunk a score of one divided by the quantity k plus its rank, summed across every retriever that returned it. The constant k , conventionally 60, damps the influence of the top few positions so a single retriever cannot dominate. Because the formula uses rank rather than score, it sidesteps the problem that cosine similarity and BM25 scores are on incomparable scales and cannot be added or averaged directly.

What reranking adds

Fusion produces good recall and mediocre precision. A cross-encoder reranker fixes the second half. Where a bi-encoder embeds the query and the chunk separately, a cross-encoder reads them together in a single forward pass and outputs a relevance score. It is far more accurate and far too slow to run over an entire corpus, which is why it belongs at the end of the funnel: retrieve forty candidates cheaply, score those forty expensively, keep five.

The scores a cross-encoder emits are unbounded logits, not probabilities. A score of 7.4 is high and a score of -1.6 is low, but the boundary between them is model-specific and shifts with the query. Treating a raw logit as a confidence value is a common mistake.

Chunking

Chunk size trades recall against precision. Large chunks preserve context and dilute the signal: the embedding of a two-thousand-character passage is an average of everything in it, so a single relevant sentence barely moves the vector. Small chunks sharpen the signal and lose the surrounding argument.

Overlap exists so that a fact spanning a boundary is not cut in half. An overlap of roughly one eighth of the chunk size is a reasonable default. Eight hundred characters with a hundred of overlap works well for prose.

Notes on local inference

Memory budgets

A model's memory footprint at inference is roughly its parameter count times the bytes per parameter, plus the key-value cache for the context window, plus whatever the runtime allocates for activations.

A 1.5-billion-parameter model at 4-bit quantization needs about 0.9 GB for weights. The same model at 16-bit needs about 3 GB. A 7-billion-parameter model at 4-bit needs about 4 GB. On a machine with 16 GB of system memory and no discrete GPU, a 7B model in 4-bit is comfortable and a 13B model is not.

The embedding model is a rounding error by comparison. all-MiniLM-L6-v2 has 22 million parameters and occupies about 90 MB on disk. The cross-encoder reranker ms-marco-MiniLM-L-6-v2 is a similar size.

Where the time goes

On CPU, in a typical local pipeline, the cost is distributed very unevenly. Indexing dominates the first run and generation dominates every run after it.

Embedding a six-hundred-page book once takes several minutes. Every query afterwards embeds a single sentence, which takes a few milliseconds. This asymmetry is the entire argument for persisting the vector index to disk: an in-memory store re-pays the multi-minute cost on every process restart, for no benefit.

BM25 index construction is fast, on the order of a second for a few thousand chunks, because it is a term-frequency count and not a neural forward pass. Querying it is faster still.

Reranking forty candidates with a small cross-encoder takes roughly 0.3 to 0.5 seconds on a modern CPU. Generating a few hundred tokens with a 1.5B model takes 15 to 30 seconds. Generation is the bottleneck, by an order of magnitude, and it is the only stage where a GPU changes the experience qualitatively.

Quantization

Quantization reduces the precision of a model's weights. Q4_K_M, the most common 4-bit format, keeps some tensors at higher precision where accuracy is most sensitive. Quality loss relative to 16-bit is small for most tasks and becomes noticeable on multi-step reasoning.

For retrieval-augmented generation specifically, quantization is more forgiving than it is for open-ended generation, because the model is summarizing supplied text rather than recalling facts from its own weights.

Reasoning models

Models in the DeepSeek-R1 family emit a chain of thought wrapped in <think> tags before their actual answer. This text is not part of the response and must be stripped before display. Failing to strip it produces the characteristic failure where a user reads the model's internal deliberation, including its false starts, as though it were the answer.